Open research with AI agents

Using today's most capable models on hard, open questions without losing control

Amir Joudaki

October 12, 2026

01 · THE PROBLEM

Smart models fail beautifully.

1 We're handing more and more research to very capable models.

2 Research fails silently: the prose, the figures, even the math look great, while one hidden assumption sinks it.

3 And there's far more output than we can check. The bottleneck becomes you.

How do you raise the signal-to-noise ratio, and stay in control?



A few gems, a lot of hay.

02 · THE BASICS

Most of the value is in the basics.

Not my invention. It's the 80/20: just learn these.

A coding agent, not a chat

Codex or Claude Code inside your repo: papers, code and results already in context.

One always-on machine

Everything in one place. Connect from anywhere; agents work overnight.

Skills

Instructions the agent loads only when a task needs them.

AGENTS.md / CLAUDE.md

Project rules, read at the start of every session.

Routing

Reference files opened only when needed.

Subagents

Fresh agents with a clean context. More on the next slide.

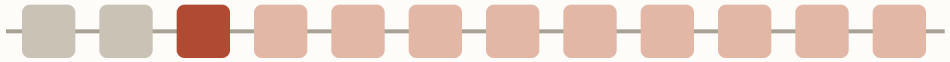
03 · MANY AGENTS

Many agents beat one long thread.

Built into `investigate-conjectures` `organize-research-project`

Not for speed: same tokens, same rate limits. Ask ten people once, not one person ten times.

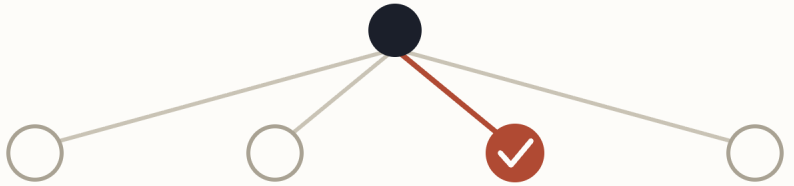
ONE LONG THREAD



one mistake ...and it never recovers

Everything piles up in one context. Once something goes wrong, everything after it inherits the mistake.

MANY FRESH AGENTS



missed missed **caught it** missed

Each starts clean and sees only its own folder. Their errors are independent, so it only takes one to catch the flaw.

04 · TWO SCALES

One question, or a whole project.

One question

a single, targeted point

FORMULATE

`investigate-conjectures`

A vague question → a sharp conjecture

SOLVE

`solve-math-rigorously`

A formal question → a rigorous solution

↑ every thrust in a project runs this pair

A whole project

one big, open question

DEVELOP

`organize-research-project`

Many thrusts → one trusted core

AUDIT

`review-ai-paper`

A finished paper → a cold, rigorous audit

↻ a review's findings can start the next round

05 · ONE QUESTION

Formulate first, then solve.

Even many agents downgrade an open question, drift from it, and lose it between hand-offs.

1 · FORMULATE

`investigate-conjectures`

Precise: freeze it as a contract.

Vague: formulate it, faithful to the spirit, and check it for ways to cheat.

Before a claim counts: separate checks rebuild every step and hunt for counterexamples.

2 · SOLVE

`solve-math-rigorously`

Only once the question is locked.

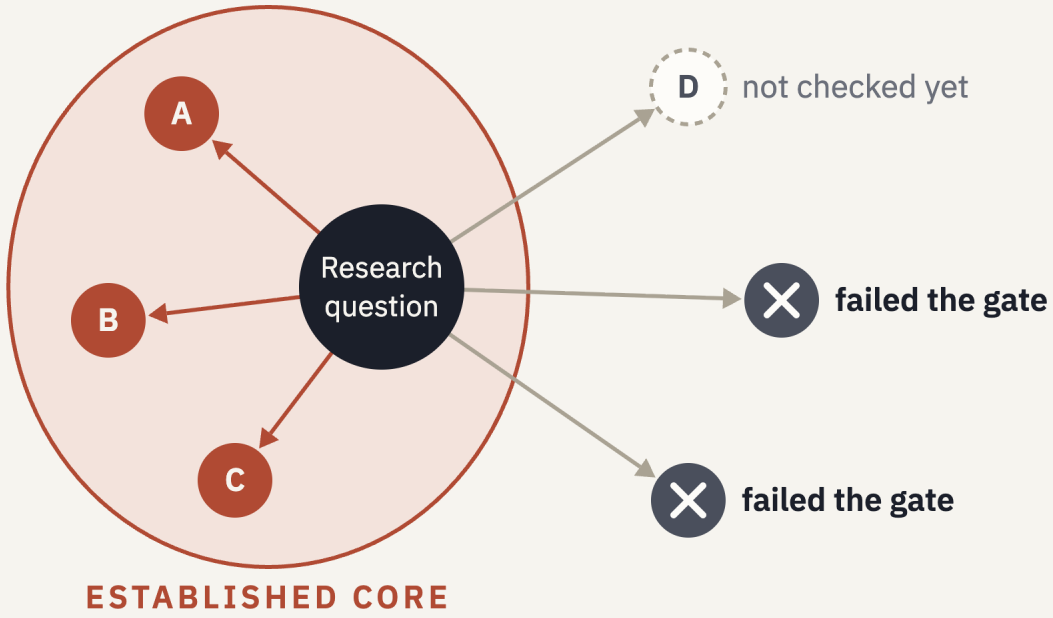
The strongest reading, never a straw man.

Every step shown; every cited theorem's assumptions checked.

Ways to cheat it checks for: an easier target · a weakened “for all” · a proxy · one example for a class

06 · A WHOLE PROJECT

Develop in isolation, then audit cold.



Inside each study: the one-question pair.

DEVELOP `organize-research-project`

The promotion gate

1 Independent audit

2 Integration: notation, tests, interfaces

3 Your approval

Pass: joins the core. ✕ Fail: stays outside, evidence kept.

AUDIT `review-ai-paper`

A fresh reviewer gets the finished paper, not its history.

07 · YOUR OWN HEAD

Make them explain it to you.

THE PROBLEM

Let $\Psi_{i\,j}^{(k)} := \Sigma_l\, \kappa_l \cdot \Lambda_{l\,j}\,(\Omega_i \otimes \xi^{(k-1)}) - \theta_j$, where κ, Λ, Ω, ξ are as in (3.7′), θ is the A.4(ii)-regularized residual, and $\partial_t \Psi = \Gamma(\Psi, \Psi')$ whenever $\|\Lambda\| < \varepsilon_0$.

Dozens of new symbols, packed into a line that takes pages to unpack.

`teach-technical-math`

A patient teacher

Teaches a paper, or the agents' own report, one block at a time.

Every symbol defined before it's used.

Confused? It rebuilds the whole explanation, not a one-line patch.

Write skills that teach the model how to talk to you.

08 · CONCLUSION

Simple ideas, a big difference, and just the beginning.

There's much more to exploit. Take what helps, add your own, and tell me your tricks.

Get the basics right	coding agent · one machine · skills · subagents
Many agents, not one thread	`investigate-conjectures` · `organize-research-project`
One question: formulate, then solve	`investigate-conjectures` → `solve-math-rigorously`
A whole project: develop, then audit	`organize-research-project` → `review-ai-paper`
Make it readable	`teach-technical-math`

github.com/ajoudaki/personal-skills